

Combinatorial mathematics

lanos212



前言

- 有微量的生成函数内容！
- 组合意义好！（当然这不代表你不应该掌握繁琐的推式）
- 杂项是想到啥讲啥，顺序很诡异请见谅。

怎么有人还要听组合恒等式，是不是被钓了

$$\binom{n}{m} = \binom{n}{n-m}$$

n 个物中选 m 个，相当于 n 个物中不选 $n-m$ 个。

$$\binom{n}{m} = \frac{n}{m} \binom{n-1}{m-1}$$

n 个物中选 m 个，第 1 个物品被选的概率为 $\frac{m}{n}$ 。

$$\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$$

n 个物中选 m 个，分讨第 1 个物品选或不选。



怎么有人还要听组合恒等式，是不是被钓了

$$\sum_{i=0}^n \binom{n}{i} = 2^n$$

每个物选或不选，等价于可以枚举选几个物然后加起来。

$$\sum_{i=0}^n (-1)^i \binom{n}{i} = [n = 0]$$

对于 $n = 0$ 显然成立，其余在杨辉三角上将一行奇数/偶数位置往上一行移，都是刚好可以填满上一行，即值都为 2^{n-1} ，故作差后为 0。

$$\sum_{i=0}^k \binom{n}{i} \binom{m}{k-i} = \binom{m+n}{k}$$

范德蒙德卷积， $n + m$ 物中选 k 个，等价于枚举 n 中选几个物然后加起来。



怎么有人还要听组合恒等式，是不是被钓了

$$\sum_{i=0}^n i \binom{n}{i} = \sum_{i=0}^n n \binom{n-1}{i-1} = n2^{n-1}$$

证明过程写出来了。

$$\sum_{i=0}^n i^2 \binom{n}{i} = \sum_{i=0}^n (i^2 + i) \binom{n}{i} = \sum_{i=0}^n n^2 \binom{n-2}{i-2} + n \binom{n-1}{i-1} = n(n+1)2^{n-2}$$

这里的 n^k 为下降幂，等价于 $n(n-1)(n-2)\cdots(n-k+1)$ 。

对于更高幂次也是一样的拆法，可能涉及后面的斯特林数。

$$\sum_{i=0}^n \binom{i}{m} = \binom{n+1}{m+1}$$

放到杨辉三角上，从上往下两两合并即可，该式称为**组合数列前缀和**。

UNOS.
= 115

怎么有人还要听组合恒等式，是不是被钓了

$$\binom{n}{r} \binom{r}{k} = \binom{n}{k} \binom{n-k}{r-k}$$

n 物中选 r 个，再从 r 个中标记 k 个，等价于先选并标记 k 个，剩下再选 $r - k$ 个。

$$\sum_{i=0}^n \binom{n-i}{i} = F_{n+1}$$

其中 F_n 为斐波那契数列第 n 项，这个可看作楼梯走 1 步或 2 步，枚举 2 步的次数。



二项式反演

如果存在 f, g 满足如下关系式:

$$f_n = \sum_{i=0}^n \binom{n}{i} g_i$$

如果知道 g , 推出 f 的每个值是平凡的。

其实反过来, 当我们知道 f 时, 我们也可以反推出 g 的每个值:

$$g_n = \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} f_i$$

这个式子称为**二项式反演**。



二项式反演

证明过程：

$$\begin{aligned} g_n &= \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} f_i \\ &= \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} \sum_{j=0}^i \binom{i}{j} g_j \end{aligned}$$

这里我们考虑单个 g_j 对 g_n 的贡献：

$$\begin{aligned} &\sum_{i=j}^n (-1)^{n-i} \binom{n}{i} \binom{i}{j} g_j \\ &= \sum_{i=j}^n (-1)^{n-i} \binom{n}{j} \binom{n-j}{i-j} g_j \end{aligned}$$

UNOS.

$$\begin{aligned}
&= \binom{n}{j} g_j \sum_{i=0}^{n-j} (-1)^{n-j-i} \binom{n-j}{i} \\
&= \binom{n}{j} g_j \cdot [n = j]
\end{aligned}$$

将该式代回，得到：

$$g_n = \sum_{j=0}^n [n = j] \cdot \binom{n}{j} g_j = g_n$$

证毕。



普通反演

如果存在 f, g 满足如下关系式:

$$f_n = \sum_{i=0}^n a_{n,i} \cdot g_i$$

如果知道 g , 推出 f 的每个值是平凡的。

其实反过来, 当我们知道 f 时, 我们也可以反推出 g 的每个值。

从小到大考虑 n 的值, $n = 0$ 时, 有式子 $f_0 = a_{0,0} \cdot g_0$, 可以推出 g_0 的值。

之后考虑到 $n = k$ 时, 假设对于 $i = 0, 2, \dots, k - 1$ 的 g_i 已经全部算出, 那么有 $f_k = \sum_{i=0}^{k-1} a_{k,i} \cdot g_i + a_{k,k} \cdot g_k$, 可以直接推得 g_k 的值。

时间复杂度 $O(n^2)$ 。



容斥原理

$$\left| \bigcup_{i=1}^n S_i \right| = \sum_{m=1}^n (-1)^{m-1} \sum_{a_i < a_{i+1}} \left| \bigcap_{i=1}^m S_{a_i} \right|$$

这是最基本的式子。

当然平时都不把这东西明写出来用，实际题目中凭经验来就行了。

比如最常用的，对满足若干条件的东西计数，那么就有：

答案 = 钦定 0 个不合法 - 钦定 1 个不合法 + ...

这里的**钦定不合法**表示，某些条件我让它一定不满足，然后别的条件都扔掉去计数。

这种都可以简单用组合恒等式证明，比如上面这个例子，

全合法的方案显然被数到 1 次，**存在不合法**的方案带着 +1 和 -1 的容斥系数被计算到的次数是一样的，所以会被抵消掉。

UNOS.
= 115

Min - Max 容斥

对于满足全序关系并且其中元素满足可加减性的序列 $\{x_i\}$, 设其长度为 n , 并设 $S = \{1, 2, 3, \dots, n\}$, 则有:

$$\max_{i \in S} x_i = \sum_{T \subseteq S} (-1)^{|T|-1} \min_{j \in T} x_j$$

$$\min_{i \in S} x_i = \sum_{T \subseteq S} (-1)^{|T|-1} \max_{j \in T} x_j$$

第一个式子证明:

考虑将 x_k 看作集合 $\{1, 2, \dots, k\}$,

那么取 max 就是集合并, 取 min 就是集合交, 发现代换掉之后和容斥原理的式子等价。

第二个反了一下, 是同理的。



Min - Max 容斥

这东西比较有用的一点是，其在期望下也成立：

$$E(\max_{i \in S} x_i) = \sum_{T \subseteq S} (-1)^{|T|-1} E(\min_{j \in T} x_j)$$

$$E(\min_{i \in S} x_i) = \sum_{T \subseteq S} (-1)^{|T|-1} E(\max_{j \in T} x_j)$$

虽然正确性是显然的，期望具有线性性，你对于所有有可能出现的情况，都把最大值/最小值容斥成一堆最小值/最大值的和，然后再乘上概率加回来，和原本的期望当然还是一样的。



Min - Max 容斥

$$\text{kthmax}_{i \in S} x_i = \sum_{T \subseteq S} (-1)^{|T|-k} \binom{|T|-1}{k-1} \min_{j \in T} x_j$$

$$\text{kthmin}_{i \in S} x_i = \sum_{T \subseteq S} (-1)^{|T|-k} \binom{|T|-1}{k-1} \max_{j \in T} x_j$$

懒得证了，[自己看](#)。

对于期望下也是成立的，式子就不写了。



Min - Max 容斥

还有变种：

$$\operatorname{lcm}_{i \in S} x_i = \prod_{T \subseteq S} \left(\operatorname{gcd}_{j \in T} x_j \right)^{(-1)^{|T|-1}}$$

证明：

相当于是对于每个质数上的指数都类似地对应成集合，
道——↗↘ 理~←→↑↓ 是一样的。



Min - Max 容斥

[HAOI2015] 按位或

Time Limit: 1 sec / Memory Limit: 128 MB

你每次随机选取一个 $[0, 2^n - 1]$ 内的整数, i 被选出的概率为 P_i 。

如果选出的所有数 or 起来为 $2^n - 1$ 就结束, 求期望选出多少个数。

如果不可能结束, 输出 INF。

数据范围

- $1 \leq n \leq 20$
- $\sum_{i=0}^{2^n-1} P_i = 1$



Min - Max 容斥

相当于求每个位变成 1 的时间之 max 的期望。

进行 min - max 容斥，变为你要求每个子集中，第一次里面出现 1 的时间的期望。

设所有在某些位都不为 1 的数字被取到的概率之和为 P ，那么这个期望就是 $\frac{1}{1-P}$ 。



Min - Max 容斥

[PKUWC2018] 随机游走

Time Limit: 1 sec / Memory Limit: 512 MB

有一棵 n 个点的树， Q 次询问如下问题：

- 每次询问给你一个集合 S 。
- 求从点 1 开始，每次等概率选一个连向的点走过去，期望多少次后经过 S 内所有点至少一次。
- 点 1 在一开始就视为经过。

每次询问的答案都对 998244353 取模。

数据范围

- $1 \leq n \leq 18$ (PKU 的机子速度，谅解一下)
- $1 \leq Q \leq 5000$



Min - Max 容斥

相当于求每个点第一次被经过的时间之 max 的期望。

min - max 容斥, 现在我们要对于每个子集计算下列问题:

- 从点 1 出发随机游走, 到达集合 S 内的点就结束, 期望走多少步。

设 f_i 表示从 i 开始随机游走, 期望多少步结束, 显然有 $f_{i \in S} = 0$ 。

对于其它的点 x , 可以写出关系式:

$$f_x = 1 + \frac{\sum_{y \in N(x)} f_y}{|N(x)|}$$

不好直接递推出每一项的值, 故设 $f_x = A \cdot f_{fa_x} + B$,

这样假设求完 x 子树内的期望, 那么 f_x 的 B 部分就可以解方程得到。

时间复杂度 $O(n \cdot 2^n \log |\text{mod}| + Q)$ 。

(log 在于求逆元, 当然用 $O(1)$ 在线逆元也可以, 常数较大)



卡特兰数

1, 1, 2, 5, 14, 42, 132, ...

这个是 OEIS 上的 [A000108](#) 数列。

这些数叫卡特兰数！

基本递推式：

$$C_0 = 1, C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i} \quad (n \geq 1)$$

相邻两项间的递推式：

$$C_0 = 1, C_n = \frac{4n-2}{n+1} C_{n-1} \quad (n \geq 1)$$



卡特兰数

常见通项表达式：

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{\binom{2n}{n}}{n+1} = \frac{\binom{2n+1}{n}}{2n+1} = \frac{(2n)!}{n!(n+1)!}$$

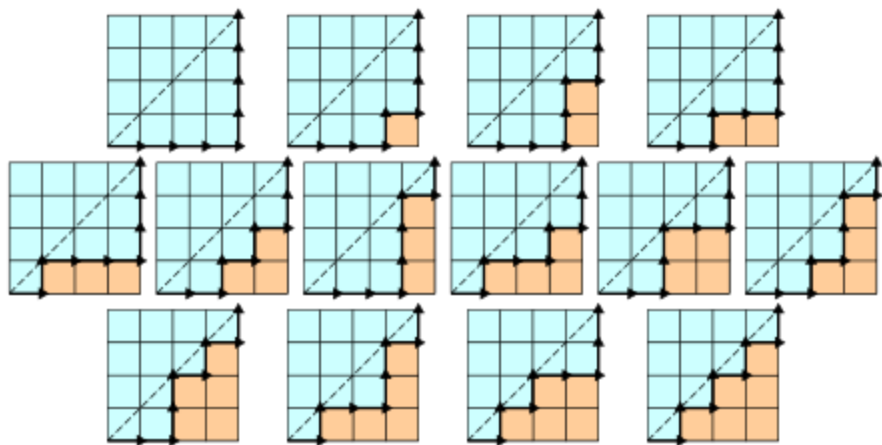
其与众多计数问题相关，如**凸多边形三角剖分数**，**合法括号序列数**，**合法出入栈序列数**，**向右/向上走不穿过对角线到达对角方案数**等等。

卡特兰数通项公式的证明方式太多了（某白猫已知的有 7 种），这里先给出一种最普遍的证明方式。



卡特兰数

使用折线图！



C_n 可以表示 $n \times n$ 的网格，从左下角走到右上角，每次只能往右或往上走，**不穿过**副对角线的方案数。

考虑 **不穿过**副对角线方案数 = 总方案数 - **穿过**副对角线方案数。

总方案数相当于 n 次向上和 n 次向右任意排布，一共有 $\binom{2n}{n}$ 种方案。



卡特兰数

我们把副对角线往上移动一格，这样子**穿过**就变成了**触碰**。

如果我们把第一次触碰后的路径全部沿着斜线对称过去，会发现，任意一条**触碰**斜线的路径都和一条到达**对称后的终点**的路径——对应。



这样的方案一共有 $\binom{2n}{n-1}$ 种。

所以就有 $C_n = \binom{2n}{n} - \binom{2n}{n-1}$ 。



卡特兰数 - 拓展 1

[ABC205 E] White and Black Balls

求有多少将 N 个白球, M 个黑球排成一行的方案 (同色球之间互不区分), 满足:

- 对于任意 i , 假设前 i 个球中有 w_i 个白球, b_i 个黑球, 那么满足 $w_i \leq b_i + K$ 。

答案对 $10^9 + 7$ 取模。

数据范围

- $0 \leq N, M \leq 10^6$
- $1 \leq N + M$
- $0 \leq K \leq N$



卡特兰数 - 拓展 1

同样使用折线图，只不过起点在折线下 K 格。

白球看作向上走，黑球看作向右走，推法是一样的。

特判 $N > M + K$ ，此时答案为 0。

其余情况类似推导得到答案为 $\binom{N+M}{N} - \binom{N+M}{N+K+1}$ 。



卡特兰数 - 拓展 2

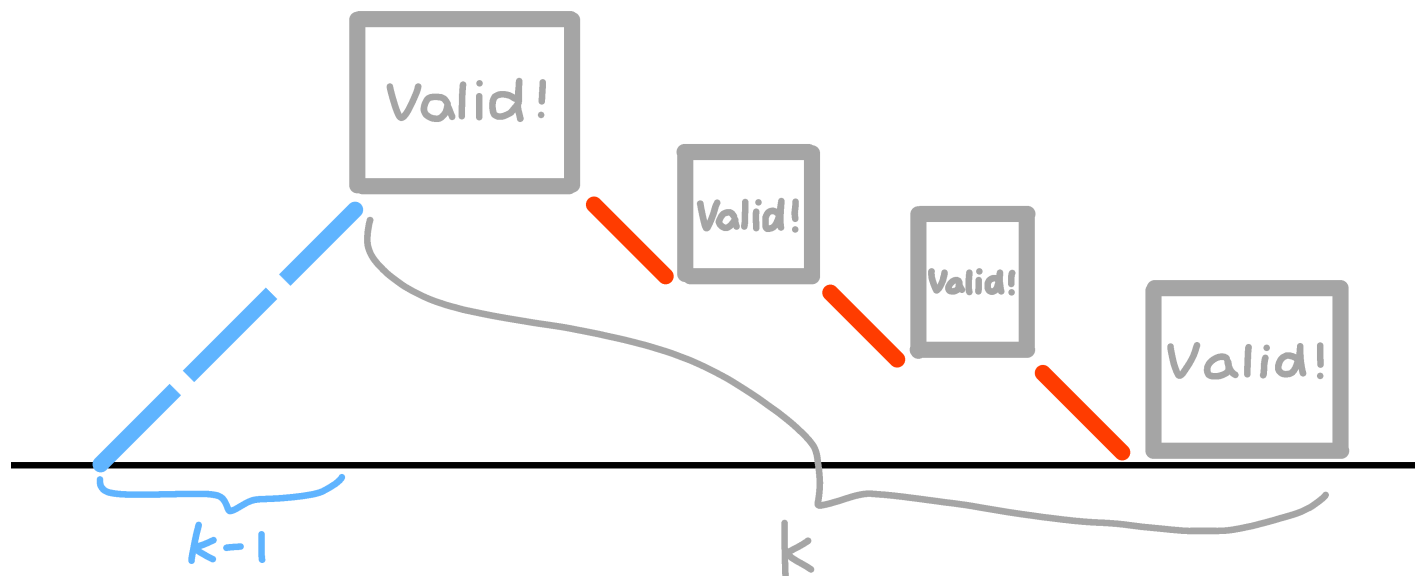
求 k 个合法括号序列，长度总和为 $2n$ 的方案数。

两种方案不同，当且仅当存在某个合法括号序列在两种方案中不同。



卡特兰数 - 拓展 2

仍然使用折线图!



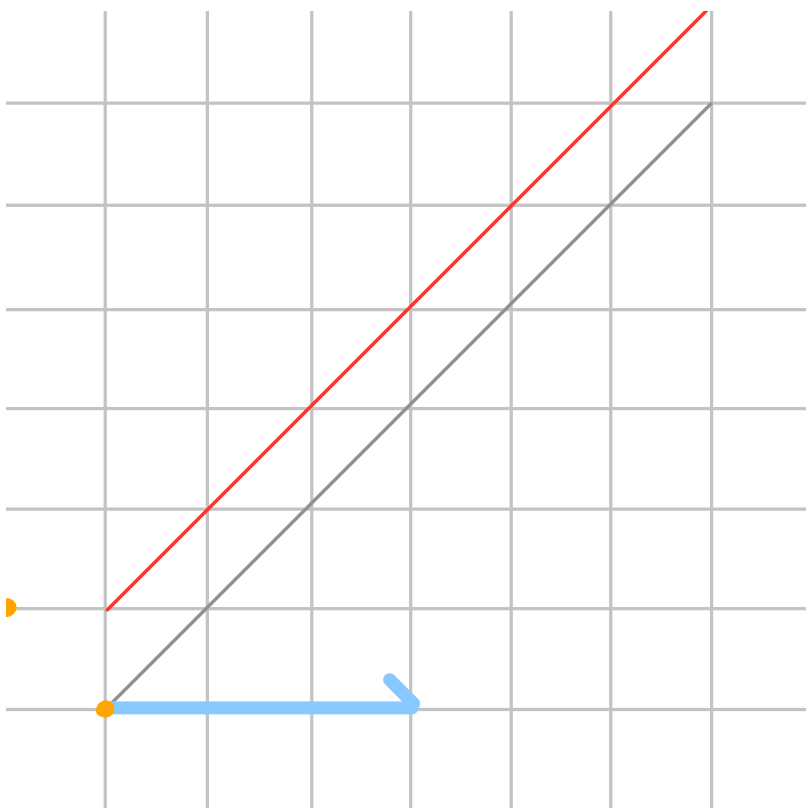
我们在最开头插入 $k - 1$ 个向上的折线，那么在这之后，
当每次折线的**最低点向下移动**，就表示我们完成了一段合法括号序列。



卡特兰数 - 拓展 2

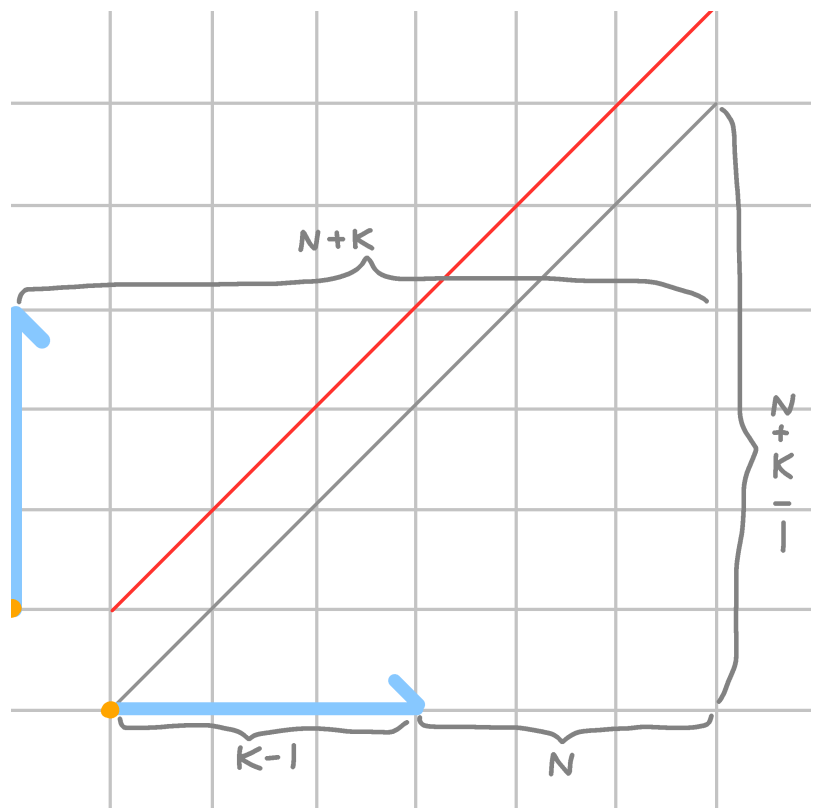
这样, k 个长度总和为 $2n$ 的合法括号序列, 就可以看作是一条在最前端钦定有 $k - 1$ 个向上的长度为 $2n + 2k - 2$ 的不穿过副对角线的折线。

(也可以看作开头钦定 $k - 1$ 个 (的长度为 $2n + 2k - 2$ 的合法括号序列)



卡特兰数 - 拓展 2

接下来的推法和 **拓展 1** 类似：



所以方案数为 $\binom{2n+k-1}{n} - \binom{2n+k-1}{n-1}$ 。



第二类斯特林数

第二类斯特林数 $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ 表示将 n 个有标号元素,

划分为 k 个互不区分的非空子集的方案数。(即放入 k 个无标号盒子, 盒子不能为空)

存在如下递推式:

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}$$

边界有 $\left\{ \begin{matrix} n \\ 0 \end{matrix} \right\} = [n=0]$ 。

递推式的证明可以考虑第 n 个元素怎么放:

- 放在之前 k 个集合中的某一个, 方案数为 $k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}$ 。
- 新开一个集合单独放第 n 个元素, 方案数为 $\left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\}$ 。



第二类斯特林数

第二类斯特林数 $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ 存在通项公式，在预处理阶乘及逆元后可以 $O(k)$ 计算。

考虑先求 n 个有标号元素放 k 个有标号盒子，盒子不能为空方案数。

考虑容斥，方案数为 钦定 0 个盒子不放 - 钦定 1 个盒子不放 + 钦定 2 个盒子不放 $\dots$

$$\sum_{i=0}^k (-1)^{k-i} i^n \binom{k}{i}$$

而第二类斯特林数的盒子是无标号的，所以还需要除以 $k!$ 去掉标号：

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \frac{\sum_{i=0}^k (-1)^{k-i} i^n \binom{k}{i}}{k!} = \sum_{i=0}^k \frac{(-1)^{k-i} i^n}{i!(k-i)!}$$



第二类斯特林数

在实际运用时，我们可以 $O(n^2)$ 递推出所有的第二类斯特林数。

运用一些多项式与生成函数的知识可以在 $O(n \log n)$ 的时间复杂度内求出某**行/列**的第二类斯特林数。



第二类斯特林数 - 行预处理

考虑如何快速求某行的第二类斯特林数，即 $\left\{ \begin{matrix} n \\ 0 \end{matrix} \right\}, \left\{ \begin{matrix} n \\ 1 \end{matrix} \right\}, \dots, \left\{ \begin{matrix} n \\ n \end{matrix} \right\}$ 。

将前面的通项公式搬过来：

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \sum_{i=0}^k \frac{(-1)^{k-i} i^n}{i!(k-i)!}$$

注意到 n 确定时，式子内的值可以分为两部分，一部分只与 i 有关，一部分只与 $k - i$ 有关。

设有两个 n 次多项式 $A(x) = \sum_{i=0}^n \frac{i^n}{i!} x^i$, $B(x) = \sum_{i=0}^n \frac{(-1)^i}{i!} x^i$ 。

直接将 A, B 进行卷积即可做到 $O(n \log n)$ 。



第二类斯特林数 - 列预处理

考虑如何快速求某**列**的第二类斯特林数，即 $\left\{ \begin{smallmatrix} 0 \\ k \end{smallmatrix} \right\}, \left\{ \begin{smallmatrix} 1 \\ k \end{smallmatrix} \right\}, \dots, \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 。

仍然考虑盒子有标号的情况，由于元素有标号，写出单个盒子的指数型生成函数：

$$F(x) = \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots = e^x - 1$$

那么就有 $\left\{ \begin{smallmatrix} i \\ k \end{smallmatrix} \right\} = \frac{\left[\frac{x^i}{i!} \right] F^k(x)}{k!}$ ，除以 $k!$ 是为了去标号。

$O(n \log n)$ 进行多项式快速幂即可。



贝尔数

贝尔数一般记作 B_n ，其表示将 n 个有标号元素划分为**任意个**非空集合的方案数。

显然有 $B_n = \sum_{i=0}^n \left\{ \begin{matrix} n \\ i \end{matrix} \right\}$ 。

所以如果你会 $O(n^2)$ 递推第二类斯特林数，那么你就会 $O(n^2)$ 递推贝尔数。

当然也有另一种递推式：

$$B_n = \sum_{i=0}^{n-1} \binom{n-1}{i} B_i$$

证明考虑第 n 个元素如何划分：

- 前面选出 $n - 1$ 个元素放一起，方案数为 $\binom{n-1}{0} B_0$ 。
- 前面选出 $n - 2$ 个元素放一起，方案数为 $\binom{n-1}{1} B_1$ 。
- ...



贝尔数

贝尔数存在 $O(n \log n)$ 求解前 n 项的方法。

考虑前面一个盒子的指数型生成函数是 $F(x) = e^x - 1$ 。

现在我们有任意多个无标号盒子，那么相当于：

$$\sum_{i=0}^{+\infty} \frac{F^i(x)}{i!} = \exp F(x)$$

使用多项式 exp 可以 $O(n \log n)$ 求解前 n 项。



第一类斯特林数

第一类斯特林数 $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right]$ 表示将 n 个有标号元素，
划分为 k 个无标号圆排列中。

圆排列是指一个任意循环位移后也视作本质相同的排列，
比如 $[1, 2, 3, 4]$, $[4, 1, 2, 3]$, $[3, 4, 1, 2]$, $[2, 3, 4, 1]$ 都是同一个圆排列，
而 $[1, 2, 3, 4]$ 与 $[1, 2, 4, 3]$ 不同。

有如下递推式：

$$\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right] = \left[\begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right] + (n-1) \left[\begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right]$$

考虑第 n 个元素，可以给它单开一个圆排列，
或者放到之前任意一个元素的后面，即可简单证明递推式的正确性。



第一类斯特林数

第一类斯特林数**暂时没有实用的通项公式**。

在实际运用时，我们可以 $O(n^2)$ 递推出所有的第一类斯特林数。

运用一些多项式与生成函数的知识可以在 $O(n \log n)$ 的时间复杂度内求出某**行/列**的第一类斯特林数。



第一类斯特林数 - 行预处理

快速求某行的第一类斯特林数，即 $\begin{bmatrix} n \\ 0 \end{bmatrix}, \begin{bmatrix} n \\ 1 \end{bmatrix}, \dots, \begin{bmatrix} n \\ n \end{bmatrix}$ 。

考虑递推式的分析方法，第 i 个元素有 $i - 1$ 种方案让圆排列数不变，1 种方案增加。

故 n 这一行的斯特林数的生成函数为 $\prod_{i=0}^{n-1} (x + i) = \frac{(x+n-1)!}{(x-1)!} = x^{\overline{n}}$ 。

该生成函数可以暴力分治卷积可以 $O(n \log^2 n)$ 求出。

用上升幂相关做法可以 $O(n \log n)$ 求出，详见 [OI wiki 多项式平移|连续点值平移](#)。



第一类斯特林数 - 列预处理

快速求某**列**的第一类斯特林数, 即 $\begin{bmatrix} 0 \\ k \end{bmatrix}, \begin{bmatrix} 1 \\ k \end{bmatrix}, \dots, \begin{bmatrix} n \\ k \end{bmatrix}$ 。

类似第二类斯特林数, 我们写出单个圆排列的指数型生成函数:

$$F(x) = \sum_{i=0}^{+\infty} (i-1)! \frac{x^i}{i!} = \sum_{i=0}^{+\infty} \frac{x^i}{i}$$

仍然用多项式快速幂求出 $F^k(x)$, 时间复杂度 $O(n \log n)$ 。



下降幂/上升幂

下降幂定义：

$$x^{\underline{n}} = x(x-1)(x-2)\cdots(x-n+1)$$

上升幂定义：

$$x^{\overline{n}} = x(x+1)(x+2)\cdots(x+n-1)$$

这里的 x 可以是实数， n 为正整数。

特别地， $x^{\underline{0}} = x^{\overline{0}} = 1$ 。

显然上升幂和下降幂之间可以有如下两种转化：

$$\begin{aligned}x^{\underline{n}} &= (x+n-1)^{\overline{n}} \\x^{\underline{n}} &= (-1)^n (-x)^{\overline{n}}\end{aligned}$$



下降幂/上升幂

对于负整数幂次，下降幂有如下定义（下式中， n 为正整数）：

$$x^{\underline{-n}} = \frac{1}{(x+1)(x+2)\cdots(x+n)}$$

相当于是满足 $x^{\underline{n}} \times (x-n) = x^{\underline{n+1}}$ （不出现分母为 0 的情况下）。

对于上升幂也是同理的。



下降幂/上升幂

有个简单的性质：

$$\begin{aligned}x^{\underline{a+b}} &= x^{\underline{a}}(x - a)^{\underline{b}} \\x^{\overline{a+b}} &= x^{\overline{a}}(x + a)^{\overline{b}}\end{aligned}$$

然后我们可以把组合数拓展到实数域：

$$\binom{r}{n} = \frac{r^{\underline{n}}}{n!}$$

可以证明，前面所有保证下标为整数的组合恒等式在上标为实数的情况下**仍然成立**。

有**广义二项式定理**（这里的 r 为实数）：

$$(a + b)^r = \sum_{i=0}^{+\infty} \binom{r}{i} a^i b^{r-i}$$



下降幂/上升幂

下降幂和组合数相乘有很好的性质：

$$\binom{n}{k} k^{\underline{i}} = \binom{n-i}{k-i} n^{\underline{i}}$$

证明可以考虑用组合恒等式中的 $\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$ 。

m 次下降幂和有相比普通幂更加优美的公式：

$$\sum_{i=0}^n i^{\underline{m}} = \frac{(n+1)^{\underline{m+1}}}{m+1}$$

使用有限微积分或者数学归纳法即可证明。



下降幂/上升幂

还有关于下降幂/上升幂的二项式定理：

$$(a + b)^{\overline{n}} = \sum_{i=0}^n \binom{n}{i} a^{\overline{i}} b^{\overline{n-i}}$$

$$(a + b)^{\underline{n}} = \sum_{i=0}^n \binom{n}{i} a^{\underline{i}} b^{\underline{n-i}}$$



下降幂/上升幂

这里给出第一个式子的证明：

$$\begin{aligned} & \sum_{i=0}^n \binom{n}{i} a^i b^{n-i} \\ &= \sum_{i=0}^n \frac{n!}{i!(n-i)!} a^i b^{n-i} \\ &= \sum_{i=0}^n n! \binom{a}{i} \binom{b}{n-i} \\ &= n! \binom{a+b}{n} \\ &= (a+b)^n \end{aligned}$$

上升幂的话使用 $x^n = (-1)^n (-x)^{\bar{n}}$ 转下降幂即可一样证明。



下降幂/上升幂与普通幂的转化

上升幂转普通幂：

$$x^{\bar{n}} = \sum_{i=0}^n \begin{bmatrix} n \\ i \end{bmatrix} x^i$$

这个正确性是显然的，因为前面已经说明过 $x^{\bar{n}}$ 是行上第一类斯特林数的生成函数。

那么下降幂转普通幂也是类似的：

$$x^n = (-1)^n (-x)^{\bar{n}} = \sum_{i=0}^n (-1)^{n-i} \begin{bmatrix} n \\ i \end{bmatrix} x^i$$



下降幂/上升幂与普通幂的转化

普通幂转下降幂：

$$x^n = \sum_{i=0}^n \left\{ \begin{matrix} n \\ i \end{matrix} \right\} x^{\underline{i}}$$

当 x 为 $0, 1, \dots, n$ 时，可以考虑组合意义。

将 n 个有标号的元素放到 x 个有标号盒子内，等式左边是直接算，右边是枚举多少个盒子内有元素，两者是等价的。

右边的上界直接取 n 没有关系，因为在 x 为非负整数时，若 $i > x$ ，有 $x^{\underline{i}} = 0$ 。

UNOS.
= 115

下降幂/上升幂与普通幂的转化

注意到我们只证明了 x 取 $n + 1$ 个不同值时，这个式子是正确的。

接下来我们可以问一下 AK 的小狗 Fido Puppy:



x 是整数的时候都是相等的



$n+1$ 个点确定一个 n 次多项式



所以两边多项式完全相等



注意到等式左右都是一个确定的 n 次多项式。

所以 $n + 1$ 个点值相同时，可以证明 x 为任意实数时，该式也是正确的。



下降幂/上升幂与普通幂的转化

普通幂转上升幂同理：

$$x^n = \sum_{i=0}^n (-1)^n \begin{Bmatrix} n \\ i \end{Bmatrix} (-x)^{\bar{i}} = \sum_{i=0}^n (-1)^{n-i} \begin{Bmatrix} n \\ i \end{Bmatrix} x^{\bar{i}}$$

灵活运用 $x^n = (-1)^n (-x)^{\bar{n}}$ 即可。



下降幂/上升幂与普通幂的转化

P6620 [省选联考 2020 A 卷] 组合数问题

Time Limit: 1 sec / Memory Limit: 256 MB

给出正整数 n, x, p 和一个 m 次多项式 $f(k) = \sum_{i=0}^m a_i k^i$, 求下式的值:

$$\left(\sum_{k=0}^n f(k) \times x^k \times \binom{n}{k} \right) \bmod p$$

数据范围

- $1 \leq n, x, p \leq 10^9$
- $0 \leq m \leq \min(n, 1000)$
- p 不一定为质数。



下降幂/上升幂与普通幂的转化

先用组合意义找下思路。

单考虑 $x^k \times \binom{n}{k}$ 就是 n 个格子选 k 个涂色，可以涂的颜色有 x 种。

考虑枚举多项式的每一项，即只是乘一个 k^i (a_i 是小问题，是常数)，那么就是把 $1, 2, \dots, i$ 这些数再扔进已染色的格子里。

这里只有有数字的格子数量直接枚举是可以接受的，考虑对此进行枚举，写出式子：

$$\sum_{i=0}^m a_i \sum_{j=0}^i \left\{ \begin{matrix} i \\ j \end{matrix} \right\} n^{\underline{j}} \cdot x^j \cdot (x+1)^{n-j}$$

发现预处理 $n^{\underline{j}}$ 和 $x^j \cdot (x+1)^{n-j}$ 怎么就做完了，组合意义太强了。

时间复杂度 $O(m^2 + \log n)$ 。

很多形如 n^k 的贡献在组合意义下多半和**第二类斯特林数**有关。



下降幂/上升幂与普通幂的转化

你也可以用惊人的组合数学直觉/经验进行推式子。

$$\begin{aligned} & \sum_{i=0}^m a_i \sum_{k=0}^n k^{\underline{i}} \cdot x^k \cdot \binom{n}{k} \\ &= \sum_{i=0}^m a_i \sum_{k=0}^n \sum_{j=0}^i \left\{ \begin{matrix} i \\ j \end{matrix} \right\} k^{\underline{j}} \cdot x^k \cdot \binom{n}{k} \\ &= \sum_{i=0}^m a_i \sum_{k=0}^n \sum_{j=0}^i \left\{ \begin{matrix} i \\ j \end{matrix} \right\} n^{\underline{j}} \cdot x^j \cdot x^{k-j} \cdot \binom{n-j}{k-j} \\ &= \sum_{i=0}^m a_i \sum_{j=0}^i \left\{ \begin{matrix} i \\ j \end{matrix} \right\} n^{\underline{j}} \cdot x^j \cdot (x+1)^{n-j} \end{aligned}$$

主要用到组合数乘下降幂的优秀性质，整个过程和组合意义是本质相同的。



Prüfer 序列

一个有标号 n 个点的树可以与一个长度为 $n - 2$, 值域在 $[1, n]$ 内的整数序列**双射**。

对于一颗有标号 n 个点的树, 我们称其双射到的上述序列为**Prüfer 序列**。

也就是说, 有标号 n 个点的树的形态有 n^{n-2} 种, 如果是有根树那就是 n^{n-1} 种。

(我们不考虑只有 1 个点的树)



树 -> Prüfer

给定树，Prüfer 序列的构建过程为执行 $n - 2$ 次如下操作：

- 选择编号最小的叶结点并删掉它，然后在序列中记录下它**连接到的**那个结点编号。

显然可以用堆维护做到 $O(n \log n)$ 。



树 -> Prüfer

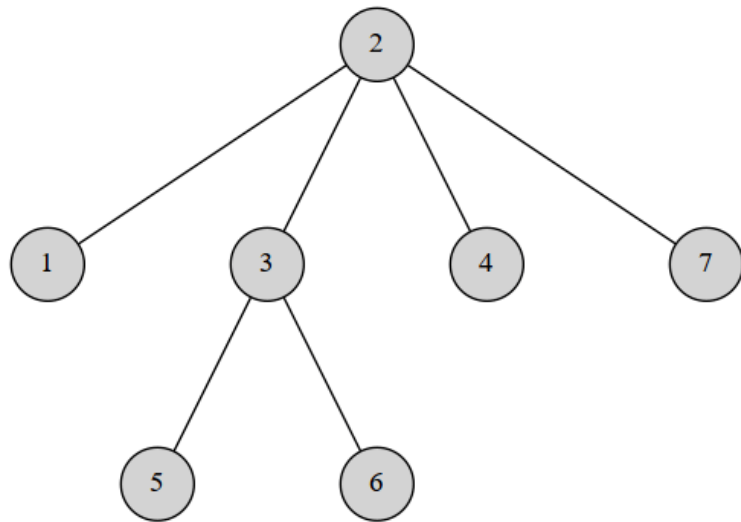
有 $O(n)$ 的构建方法, 需要维护一个指针 p , 最初指向编号最小的叶节点。

不断执行下列操作, 中途若删除了 $n - 2$ 个点则立刻结束:

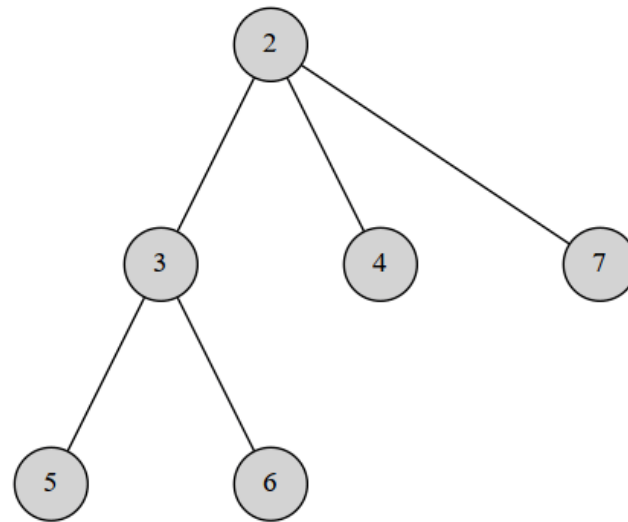
- 删除 p 指向的结点, 并检查是否产生新的叶结点。
- 如果产生新的叶结点, 假设编号为 x 。如果 $x > p$, 那么不做其他操作; 否则立刻删除 x , 然后继续检查删除 x 后是否产生新的叶结点, 以此类推, 直到未产生新节点或者新节点的编号 $> p$ 。
- 让指针 p 自增直到遇到一个未被删除叶结点为止。



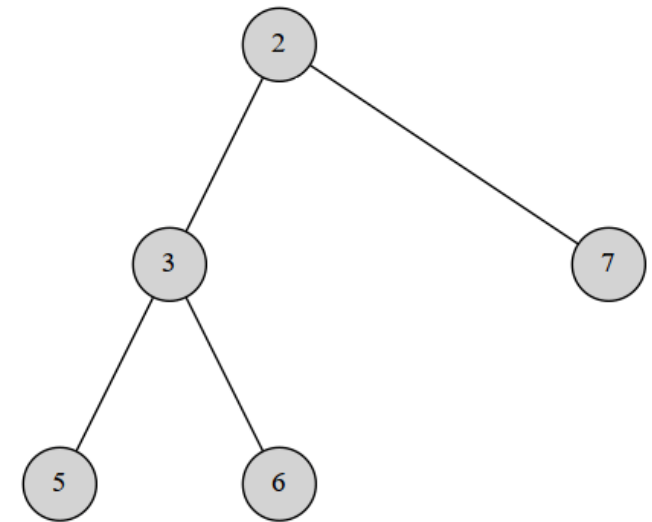
树 -> Prüfer



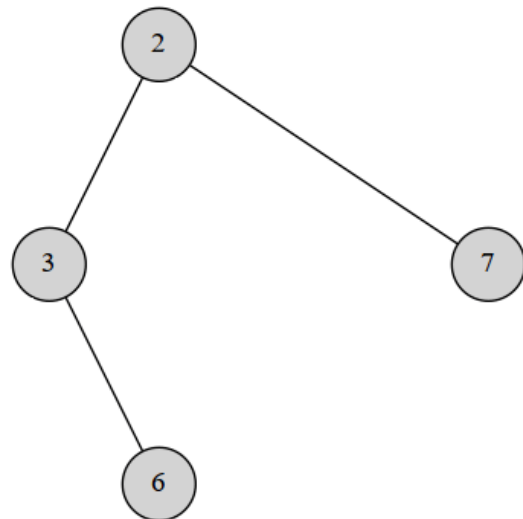
Prüfer Code:



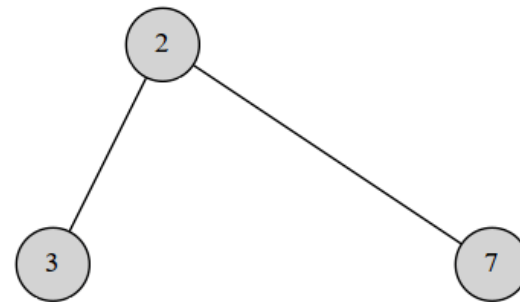
Prüfer Code: 2



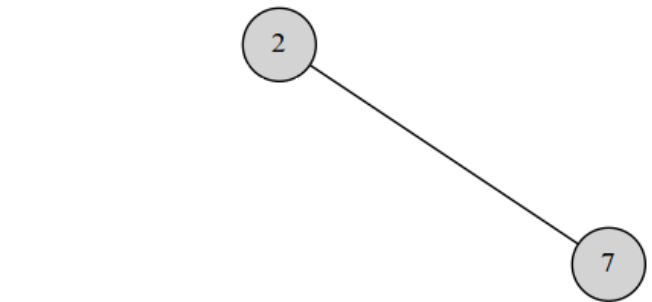
Prüfer Code: 2 2



Prüfer Code: 2 2 3



Prüfer Code: 2 2 3 3



Prüfer Code: 2 2 3 3 2

Prüfer 序列性质

- 在构造完 Prüfer 序列后原树中会剩下两个结点，其中一个一定是编号最大的点 n 。
- 每个结点在序列中出现的次数是其度数减 1。（没有出现的就是叶结点）



Prüfer -> 树

根据 Prüfer 序列的性质，先算出每个点的度数，按照 i 从 1 到 $n - 2$ 执行如下流程：

- 找度数为 1 的编号最小的未删除点，设其为 x 。
- 将 x 与 Prüfer 序列的第 i 个元素连边，删除 x ，并让连边的点度数 -1 。

最后将剩余的两个点连边即可。

可以类似 树 -> Prüfer 的方法，使用指针做到 $O(n)$ 。



Prüfer 序列

前面提到，每个结点在序列中出现的次数是其度数减 1。

那么如果说给你每个点的度数，让你求可能的有标号树数量，那么就是：

$$\binom{n-2}{d_1-1, d_2-1, \dots, d_n-1}$$

这里的 d_i 表示点 i 的度数。

注意， d_i 都应该是在 $[1, n-1]$ 内的整数，否则树的个数应该为 0。



Prüfer 序列

Prüfer 序列还可以做如下问题：

n 点 m 边的带标号无向图有 k 个连通块，第 i 个连通块大小为 s_i 。
我们希望添加 $k - 1$ 条边使得整个图连通，求方案数。



Prüfer 序列

直接设 d_i 表示第 i 个连通块连出去的度数，列出式子：

$$\sum_{\sum d_i = 2k-2, d_i \in [1, k-1], d_i \in \mathbb{N}} \binom{k-2}{d_1-1, d_2-1, \dots, d_k-1} \cdot \prod_{i=1}^k s_i^{d_i}$$

考虑用 $d_i - 1$ 表示这个式子形式会更加好一点：

$$= \prod_{i=1}^k s_i \sum_{\sum d_i - 1 = k-2, d_i - 1 \in [0, k-2], d_i - 1 \in \mathbb{N}} \binom{k-2}{d_1-1, d_2-1, \dots, d_k-1} \cdot \prod_{i=1}^k s_i^{d_i-1}$$

发现后面这个是多元二项式定理，可以直接化简：

$$= (s_1 + s_2 + \dots + s_k)^{k-2} \prod_{i=1}^k s_i = n^{k-2} \prod_{i=1}^k s_i$$



Prüfer 序列

[ARC106 F] Figures

Time Limit: 2 sec / Memory Limit: 1024 MB

有 N 个插座，第 i 个插座有 a_i 个插口。

用 $N - 1$ 条双端都是插头的线将所有插座连起来，求有多少种本质不同的方案。

两种方案不同，当且仅当存在两个插座，在一种方案中它们被一条双端都是插头的线直接连接，而在另一种方案中没有，或者都被连接但是存在某一端插入的插口不同。

答案对 998244353 取模。

数据范围

- $2 \leq N \leq 2 \times 10^5$
- $1 \leq a_i < 998244353$



Prüfer 序列

直接设 d_i 表示第 i 个插座连出去的度数，列出式子：

$$\begin{aligned} & \sum_{\sum d_i = 2N-2, d_i \in [1, N-1], d_i \in \mathbb{N}} \binom{N-2}{d_1-1, d_2-1, \dots, d_N-1} \cdot \prod_{i=1}^N a_i^{d_i} \\ &= \sum_{\sum d_i-1 = N-2, d_i-1 \in [0, N-2], d_i-1 \in \mathbb{N}} \frac{(N-2)!}{(d_1-1)!(d_2-1)! \cdots (d_N-1)!} \cdot \prod_{i=1}^N (a_i \cdot (a_i-1)^{d_i-1}) \\ &= (N-2)! \cdot \prod_{i=1}^N a_i \cdot \sum_{\sum d_i-1 = N-2, d_i-1 \in [0, N-2], d_i-1 \in \mathbb{N}} \prod_{i=1}^N \binom{a_i-1}{d_i-1} \end{aligned}$$

后面的组合意义相当于是总共 $\sum_{i=1}^N a_i - N$ 个物品中选总共 $N-2$ 个：

$$= (N-2)! \cdot \prod_{i=1}^N a_i \cdot \binom{\sum_{i=1}^N a_i - N}{N-2}$$



Prüfer 序列

$$(N - 2)! \cdot \prod_{i=1}^N a_i \cdot \binom{\sum_{i=1}^N a_i - N}{N - 2}$$
$$= \prod_{i=1}^N a_i \cdot \binom{\sum_{i=1}^N a_i - N}{N - 2}$$

直接 $O(N)$ 全部乘起来即可。



Raney 定理相关

Raney 定理:

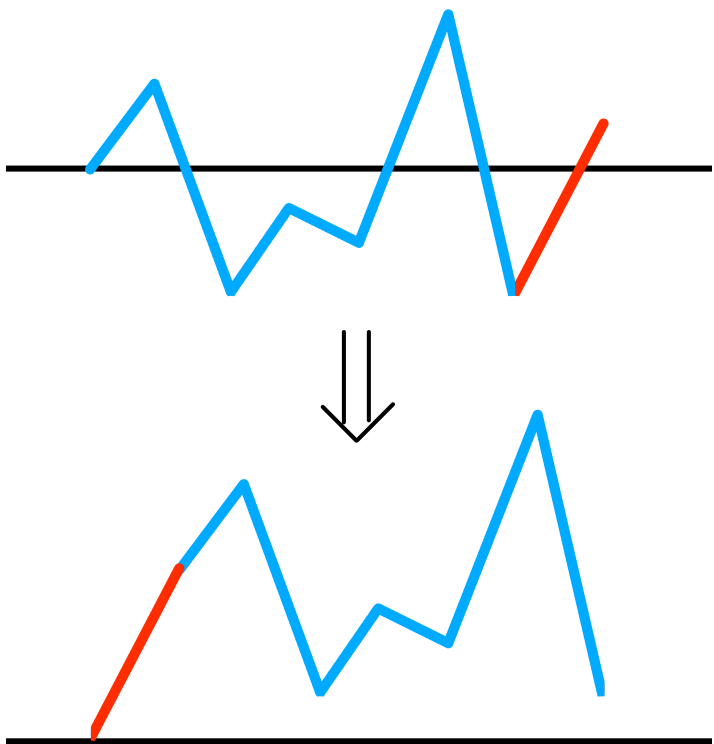
和为 1 的整数序列的所有循环位移序列中，**有且仅有一个**满足任意前缀和大于 0。

该定理比较小巧，可以用于解决卡特兰数，多边形划分等问题。



Raney 定理相关

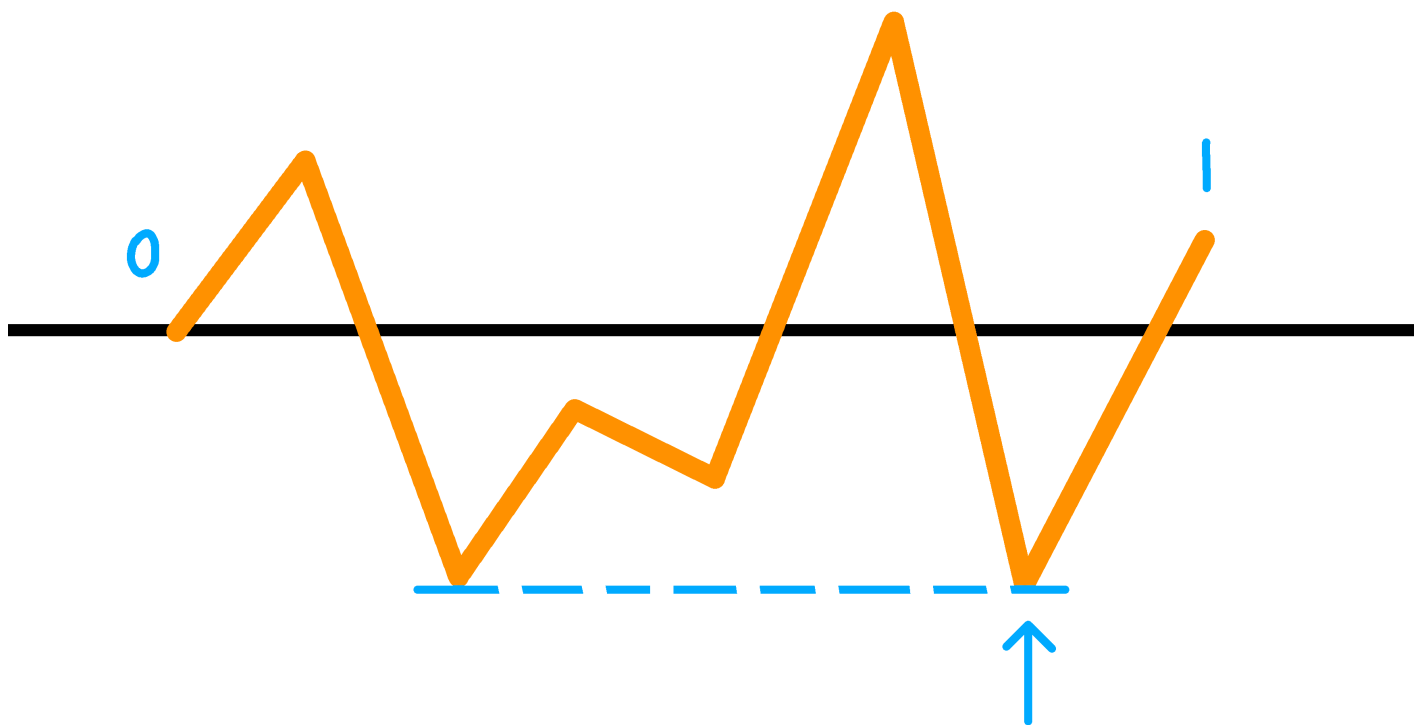
证明可以考虑将这个序列的**前缀和**用折线图表示：



此时我们将**最右侧**的折线**最低点**循环位移上来，显然满足任意前缀和大于 0。



Raney 定理相关



而其余点循环位移上来，在这个点都会让前缀和**小于等于 0**。

Raney 定理相关

关于这种序列，还有一个定理。

定理 2:

和为 1 的整数序列的所有循环位移序列**互不相同**。

证明:

考虑反证法，如果存在两个循环位移序列相同，
那么整条折线就可以被划分成 $k (k \geq 2)$ 段完全相同的折线，
使得序列总和一定是 k 的倍数，矛盾。



Raney 定理相关

这东西能干什么？

先推一个简单的卡特兰数。

考虑由 $n + 1$ 个 $+1$, n 个 -1 构成的所有长度为 $2n + 1$ 的序列中, 有 C_n 个满足序列的任意前缀和 > 0 。

总共有 $\binom{2n+1}{n+1}$ 个整数序列, 那么 $C_n = \frac{\binom{2n+1}{n+1}}{2n+1} = \frac{\binom{2n}{n}}{n+1}$ 。

这是因为所有序列可以根据循环位移后两两间是否能够变得相同来分组, 那么每组中都有恰好 $2n + 1$ 个序列, 且每组中有恰好 1 个满足条件。



Raney 定理相关

大的要来了。

给出一个顶点有标号的正 n 边形，求在顶点之间切 $k - 1$ 刀（不能交叉）

将其划分成 k 部分的方案数。

要求在预处理组合数和逆元后 $O(1)$ 查询。

目前已知的做法有二元生成函数大力推导，大量组合意义和双射转化等。

使用 Raney 定理的做法大概是目前最简洁明了的一种。



Raney 定理相关

首先凸多边形 k 划分的形式比较无序，我们按顺时针顺序，让后面的点连向前面的点。

那么考虑按顺时针标号，从 1 号点开始，可以抽象成两种操作：

- 将当前点入栈，移动到下一个点。
- 弹出栈中至少一个元素（不能弹空），然后向栈顶连边。

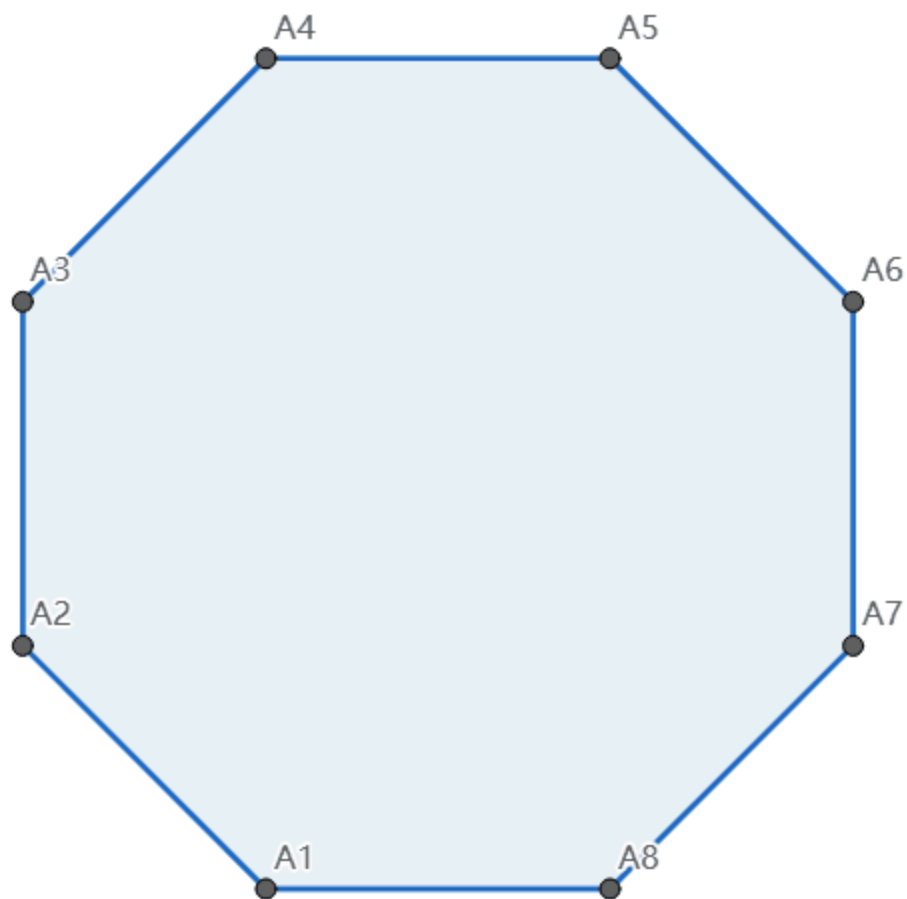
最后我们强制让栈内弹到只剩一个元素，然后把 1 和 n 之间那条多余边去掉。

这个时候用一个整数序列记录栈内元素增减情况，入栈为 $+1$ ，弹出 p 个为 $-p$ ，那么整个序列就是一个总和为 1 的整数序列。



Raney 定理相关

(若无法渲染, 点击 GIF 打开)



空

Raney 定理相关

可以分析得到凸多边形 k 划分与由 $n - 1$ 个 $+1$, k 个负数组成的总和为 1 的整数序列双射。

将 $-(n - 2)$ 分配给 k 个负整数的方案数可以使用插板法, 计算得到 $\binom{n-3}{k-1}$ 。

故凸多边形 k 划分的方案数为:

$$\frac{1}{n + k - 1} \binom{n + k - 1}{k} \binom{n - 3}{k - 1}$$



Raney 定理相关

可以分析得到凸多边形 k 划分与由 $n - 1$ 个 $+1$, k 个负数组成的总和为 1 的整数序列双射。

将 $-(n - 2)$ 分配给 k 个负整数的方案数可以使用插板法, 计算得到 $\binom{n-3}{k-1}$ 。

故凸多边形 k 划分的方案数为:

$$\frac{1}{n + k - 1} \binom{n + k - 1}{k} \binom{n - 3}{k - 1}$$



Raney 定理相关

[AGC 065 D] Not Intersect

Time Limit: 3 sec / Memory Limit: 1024 MB

给你一个圆环，上面有 N 个点，
问在点与点之间连 M 条不相交或只在顶点相交的线段的方案数，对 $10^9 + 7$ 取模。

数据范围

- $1 \leq N \leq 10^7$
- $0 \leq M \leq \frac{N(N-1)}{2}$



Raney 定理相关

特判 $N \leq 2$ 。

其余情况点按照顺时针顺序连起来显然是个凸多边形。

特判 $M > 2N - 3$ 时无解，因为内部划分最多 $N - 3$ 条，边上最多 N 条。

然后枚举凸多边形上的边连多少条，里面用凸多边形 k 划分计算。

时间复杂度 $O(N)$ 。



Raney 定理相关

特判 $N \leq 2$ 。

其余情况点按照顺时针顺序连起来显然是个凸多边形。

特判 $M > 2N - 3$ 时无解，因为内部划分最多 $N - 3$ 条，边上最多 N 条。

然后枚举凸多边形上的边连多少条，里面用凸多边形 k 划分计算。

时间复杂度 $O(N)$ 。



分拆数

分拆数是指将 n 拆分成若干个无序正整数之和的方案数，这里记作 $p(n)$ 。

如对于 4 的拆分有如下几种：

- 4
- $1 + 3$
- $2 + 2$
- $1 + 1 + 2$
- $1 + 1 + 1 + 1$

故有 $p(4) = 5$ 。



分拆数

考虑如何快速算出 $p(n)$ 。

我们设一个阈值 m ，然后把拆出来的数字分成 $< m$ 和 $\geq m$ 两部分计算。

设第一部分和为 i 的方案数为 f_i ，第二部分和为 i 的方案数为 g_i ，

那么就有 $p(n) = \sum_{i=0}^n f_i g_{n-i}$ 。



分拆数

第一部分可以直接枚举当前考虑拆出来的数字，
然后是一个完全背包问题，时间复杂度 $O(nm)$ 。

第二部分考虑你是在做这样两个操作：

- 拆出来一个数 m 。
- 将之前所有拆出来的数都 $+1$ 。

用 $dp_{i,j}$ 表示，当前拆出来了 i 个数，拆出来的总和为 j 的方案数，
由于 $im \leq n$ ，所以时间复杂度为 $O(\frac{n^2}{m})$ 。

取 $m = \lfloor \sqrt{n} \rfloor$ ，那么总时间复杂度为 $O(n\sqrt{n})$ 。

如果要算出 $p(1), p(2), \dots, p(n)$ ，
直接将 $dp_{0,i}$ 初始赋为 f_i 即可，这样就不用合并 f, g 了。



分拆数

实际上存在更加优秀的做法。

考虑这样一个函数：

$$\Phi(x) = \prod_{n=1}^{+\infty} (1 - x^n)$$

可以证明，其满足这样一个式子：

$$\Phi(x) = \prod_{n=1}^{+\infty} (1 - x^n) = 1 + \sum_{n=1}^{+\infty} (-1)^n x^{\frac{n(3n\pm 1)}{2}}$$

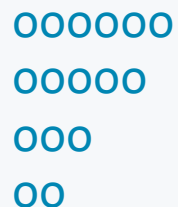
即 $\Phi(x)$ 的任意次项的系数只可能为 $-1, 0, 1$ ，
且只有在形如 $\frac{n(3n\pm 1)}{2}$ 次项的系数不为零。



分拆数

根据定义 $\Phi(x) = \prod_{n=1}^{+\infty} (1 - x^n)$, 那么展开后, 其 x^n 项的系数就为
将 n 拆成偶数个互不相同的数的方案数 — 将 n 拆成奇数个互不相同的数的方案数。

考虑这样的图形:



```
oooooo
ooooo
ooo
oo
```

其中每一行  的个数表示一个分拆出来的数, 越大的数放在越上面。

比如这个图形表示 $16 = 6 + 5 + 3 + 2$ 这个拆分。

注意这里的组合意义和分拆数定义不同, 这里拆出的数要**互不相同**。



分拆数

我们考虑将大部分**偶数方案**和**奇数方案**进行抵消。

设 m 为最后一行的 $\circ$ 个数, n 为最右侧的 45° 斜线上的 $\circ$ 个数。

```
ooooon
ooooon
ooo
mm
```

如上图中的 $m = 2, n = 2$ 。



分拆数

如果 $m \leq n$, 我们将这些 m 分别放到前若干行的末尾:

ooooon	->	ooooonm
ooooon		ooooonm
ooo		ooo
mm		

如果 $m > n$, 我们将这些 n 全部放到最后一行下面一行:

oooooon	->	oooooo
ooooon		ooooo
mmm		mmm
		nn

对于大部分情况, **偶数方案**和**奇数方案**是可以一一对应抵消的, 所以我们只要考虑不能一一对应的情况就可以了。



分拆数

可以发现，只有以下两种情况会导致无法一一对应。（下图中 x 表示 m ， n 重叠）

对于 $n = m$ ，且存在重合的情况，会导致最后一行放过去放不够：

oooo	->	oooo
nm		nm
mm		x

对于 $n = m + 1$ ，且存在重合的情况，放过去会导致出现两行长度相同：

oooo	->	oooo
nm		nm
mm		mm
mm		nn

前者有 $\frac{m(3m-1)}{2}$ 个格子，后者有 $\frac{m(3m+1)}{2}$ 个格子。



分拆数

故对于某个正整数 m , $\Phi(x)$ 在 $x^{\frac{m(3m\pm 1)}{2}}$ 这一项的系数为 $(-1)^m$ 。

也就是推得了前面的那个式子:

$$\Phi(x) = \prod_{n=1}^{+\infty} (1 - x^n) = 1 + \sum_{n=1}^{+\infty} (-1)^n x^{\frac{n(3n\pm 1)}{2}}$$



分拆数

我们考虑一下分拆数的生成函数：

$$P(x) = \prod_{n=1}^{+\infty} \frac{1}{1 - x^n}$$

这个是比较直观的，因为每个正整数都可以取无限个。

再观察一下，发现这不就是 $\frac{1}{\Phi(x)}$ 。

由于 $\Phi(x)$ 在不高于 n 次项中，只有 $O(\sqrt{n})$ 项系数不为 0，所以直接对多项式暴力求逆是 $O(n\sqrt{n})$ 的，代码极短且常数极小。

当然也可以使用多项式求逆做到 $O(n \log n)$ 。



分拆数

[NOI Online #1 入门组] 跑步

Time Limit: 2 sec / Memory Limit: 256 MB

给出 n, P , 求出 $p(n) \bmod P$ 。

数据范围

- $1 \leq n \leq 10^5$
- $1 \leq P < 2^{30}$

直接做就行了，DP 或者暴力求逆都是 $O(n\sqrt{n})$ 的，
个人建议后者，从时间空间以及代码实现上都优于前者。
模数是读入的，这里不方便上多项式科技。



分拆数

[The 3rd Universal Cup. Stage 19: Shenyang A] Safety First

Time Limit: 3 sec / Memory Limit: 1024 MB

给定 n, m , 你需要对于所有长度为 n 且单调不升的正整数序列 A , 求出下列操作的方案数, 然后求和:

- 选出一个子集, 里面必须包含第 1 个元素, 且元素和为 m 。

两种方案不同, 当且仅当存在第 i 个元素, 其在一种方案中被选, 而另一种没有。

T 组数据, 答案对 998244353 取模。

数据范围

- $1 \leq T \leq 10^5$
- $1 \leq n, m \leq 2000$



分拆数

考虑用前面的分段 DP 方法。

设一个阈值 k , 对 $< k$ 和 $\geq k$ 的数分开 DP。

对于 $< k$ 的数, 直接做完全背包, $f_{i,j}$ 记录有多少数以及被选数之和, 时间复杂度 $O(nmk)$ 。

对于 $\geq k$ 的数, 考虑每次整体 $+1$ 或加入一个 k ,

设 $dp_{i,j,k}$ 表示有多少数被选, 总共有多少数, 被选数的总和为多少。

由于 $ik \leq m$, 所以时间复杂度 $O(\frac{nm^2}{k})$ 。

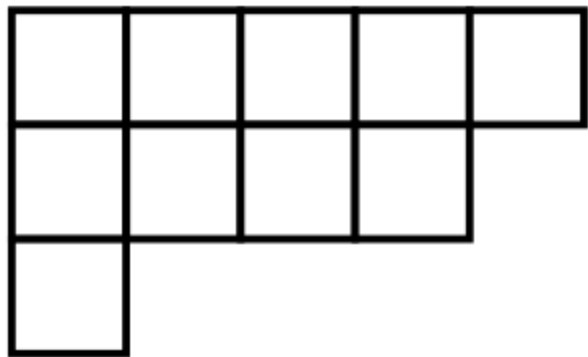
取 $k = \lfloor \sqrt{m} \rfloor$, 单组数据时间复杂度 $O(nm\sqrt{m})$ 。

同样地, 先算 f 然后把 $dp_{0,j,k}$ 初值设为 $f_{j,k}$ 即可一次算出所有答案。这样的时间复杂度就是 $O(T + nm\sqrt{m})$ 了。



杨表

杨表是一个网格图状物，其满足每行格子数**单调不升**。



对于一个 n 行的杨表，可以用 $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_n)$ 表示它，其中 λ_i 为第 i 行的格子数。

比如上面的杨表可以用 $\lambda = (5, 4, 1)$ 表示。



杨表

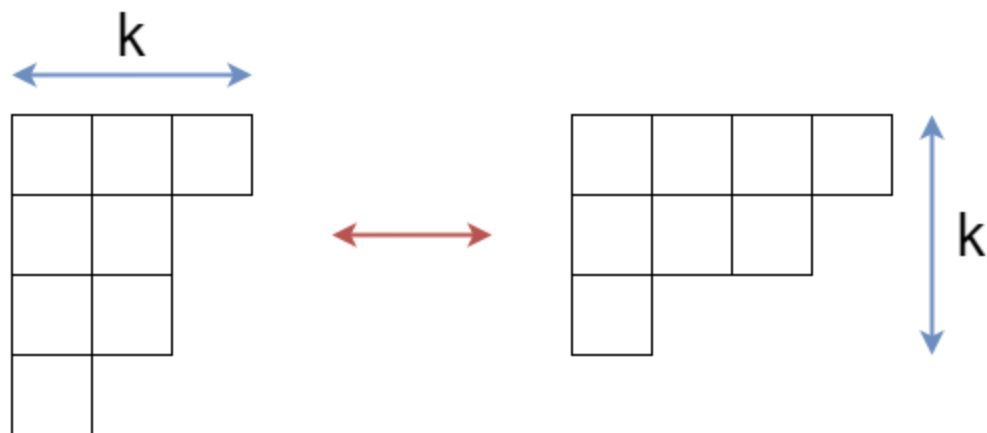
我们称一个杨表的**共轭杨表**为，将其行列进行反转后得到的杨表。



这一般用于对题意/性质的转化上。

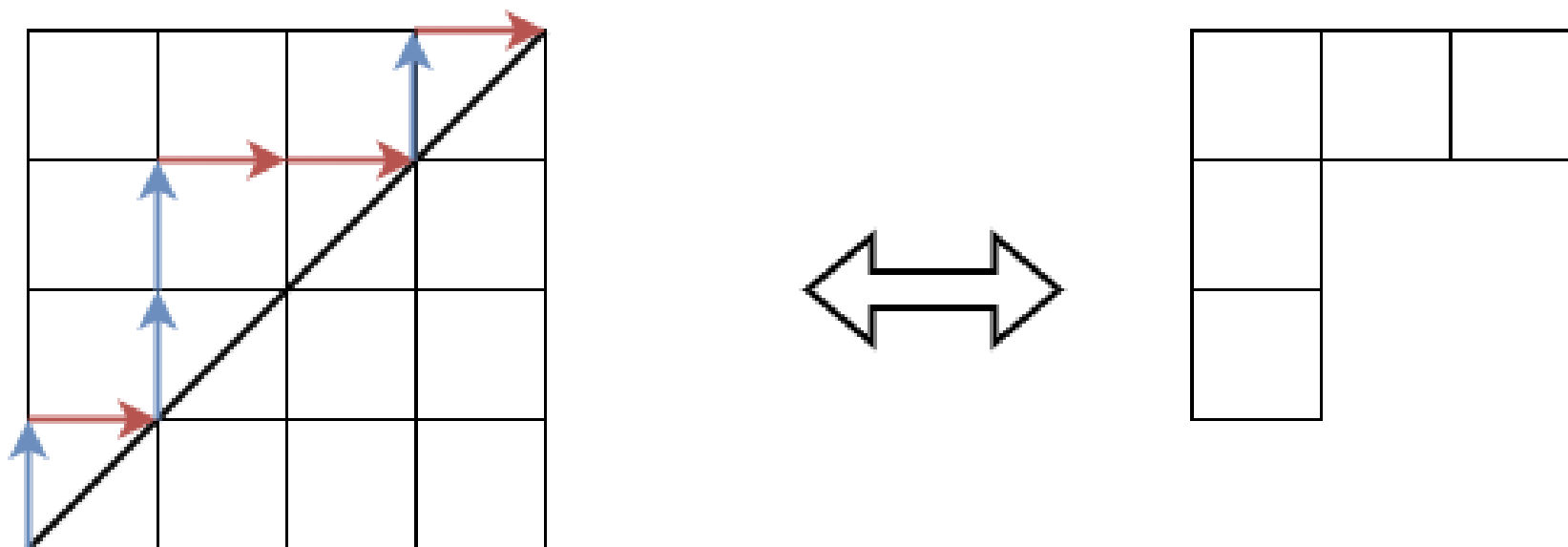
比如可以以此简单证明如下定理：

将 n 划分为若干个**不超过** k 的正整数的方案数 = 将 n 划分为 k 个正整数的方案数



杨表

杨表可以与左侧这种折线图双射。



故行数不超过 n , 列数不超过 m 的杨表个数为 $\binom{n+m}{n}$ 。

杨表

如何求总共有 n 个格子的杨表数量？

发现这就是分拆数，直接用前面分拆数的方法做就可以了。



杨表

将 $1, 2, \dots, n$ 这些数填入杨表的格子中,
这叫做 Tableau (音 [tə'blou], 这里你可以译为**数字的铺法**)。

对于一个 **标准** Tableau, 其满足如下条件:

- $1, 2, \dots, n$ 分别都恰好填入一个, 其中 n 是杨表的格子数。
- 对于上下相连的两个格子, 下方格子上的数字**大于**上方格子上的数字。
- 对于左右相连的两个格子, 右边格子上的数字**大于**左边格子上的数字。

比如下方是一个**标准** Tableau:

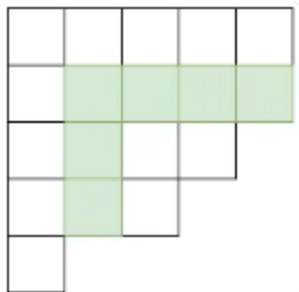
1	2	6	8
3	5		
4	7		
9			



杨表

这里将杨表第 i 行 j 列的格子记作 (i, j) 。

定义 $h(i, j)$ 为格子 (i, j) 的**钩长** (hook length),
其表示这个格子正下方和正右方, 加上自己一共有多少个格子。



对于一个杨表 λ , 其**标准** Tableau 数量可以由如下公式得到:

$$\frac{n!}{\prod_{(i,j) \in \lambda} h(i,j)}$$

其中 n 表示杨表的格子数, 这个公式被称为**钩长公式**。



杨表

该公式证明比较繁杂，不在这里写出。

[国家集训队 2019 论文集](#)与众多博客中有各种不同的证明方式，有兴趣可以自行查阅。

我们可以先拿这个公式大炮打蚊子，感受一下。

考虑一个 $\lambda = (n, n)$ 的杨表。

其的任意一个**标准** Tableau 满足任意过程中，第二行填的数个数不超过第一行。

这是一个卡特兰数的形式，那么我们就可以用钩长公式求出卡特兰数：

$$C_n = \frac{(2n)!}{\prod_{i=1}^2 \prod_{j=1}^n i + j - 1} = \frac{(2n)!}{n!(n+1)!} = \frac{\binom{2n}{n}}{n+1}$$

这里先不放例题，因为该公式往往需要搭配后面的知识。



杨表

杨表的**半标准** Tableau 定义如下:

- 将 $1, 2, \dots, n$ 填入杨表的格子中, n 为杨表的行数。
- 对于上下相连的两个格子, 下方格子上的数字**大于**上方格子上的数字。
- 对于左右相连的两个格子, 右边格子上的数字**不小于**左边格子上的数字。

这里如果你的杨表行数小于 n , 可以在最后补上若干个 0 个格子的行。

其数量存在如下公式:

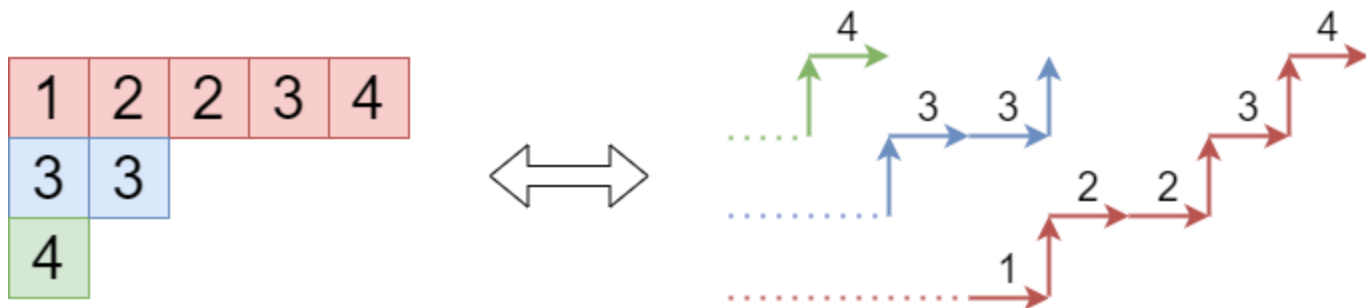
$$\prod_{1 \leq i < j \leq n} \frac{\lambda_i - \lambda_j + j - i}{j - i}$$

下面给出该公式的证明。



杨表

一个半标准 Tableau 可以与 n 条网格图上不交路径形成双射。



(本图中 $n = 4$, 也就是末尾有 0 格子的行, 右图的左上角有起点终点重叠的路径)

第 i 条路径的起点为 $(0, i)$, 终点为 $(\lambda_i + n - i, n)$ 。

根据 LGV 引理, 得到路径的总方案数为:

$$\det \left[\binom{\lambda_j + n - j + n - i}{n - i} \right]$$

这里 i 表示方阵的行, j 表示方阵的列, 标号从 1 到 n 。



杨表

$$\begin{aligned} & \det \left[\binom{\lambda_j + n - j + n - i}{n - i} \right] \\ &= \prod_{i=0}^{n-1} \frac{1}{i!} \det [(\lambda_j + n - j)^{\overline{n-i}}] \\ &= \prod_{i=0}^{n-1} \frac{1}{i!} \det [(\lambda_j + n - j)^{n-i}] \end{aligned}$$

其中第二个等号可以通过若干次矩阵初等变换得到。

现在得到了一个范德蒙德矩阵，其行列式存在一个简单的公式。

$$= \prod_{i=0}^{n-1} \frac{1}{i!} \prod_{1 \leq i < j \leq n} (\lambda_i - \lambda_j + j - i) = \prod_{1 \leq i < j \leq n} \frac{\lambda_i - \lambda_j + j - i}{j - i}$$

杨表

[Yukicoder No.2556] Increasing Matrix

Time Limit: 6 sec / Memory Limit: 512 MB

有一个 $N \times N$ 大小的网格图，求满足如下规则的填数方案数：

- 对于上下相连的两个格子，下方格子上的数字**不小于**上方格子上的数字。
- 对于左右相连的两个格子，右边格子上的数字**不小于**左边格子上的数字。
- 格子 (i, i) 上的数为 A_i 。
- 填入的数字都为正整数。

答案对 998244353 取模。

数据范围

- $1 \leq N \leq 10^5$
- $1 \leq A_1 \leq A_2 \leq \dots \leq A_N \leq 10^9$



杨表

由于主对角线右上方和左下方两部分是一样的，这里只考虑右上方的计算方法。

考虑将从右上角开始到主对角线的每个 45° 斜排看作一个杨表，
那么这是 $\lambda = (A_N, A_{N-1}, \dots, A_1)$ 这个杨表慢慢 "长出来" 的过程。

考虑将第 i 阶段出现的格子标上 i ，
可以证明这是一个与题意中内容双射的杨表半标准 Tableau。

比如 $\begin{pmatrix} 1 & 1 & 1 & 2 \\ & 2 & 3 & 3 \\ & & 3 & 3 \\ & & & 4 \end{pmatrix}$ 这种填法，可以与如下半标准 Tableau 双射：

1	1	2	4
2	3	3	
3	4		
4			



杨表

那么答案就是 $\lambda = (A_N, A_{N-1}, \dots, A_1)$ 这个杨表半标准 Tableau 数的平方。

代入半标准 Tableau 公式：

$$\left(\prod_{1 \leq i < j \leq n} \frac{A_i - A_j + j - i}{j - i} \right)^2$$

将分母提出单独计算：

$$\prod_{i=0}^{n-1} \frac{1}{(i!)^2} \prod_{1 \leq i, j \leq n, i \neq j} (A_i - A_j + j - i)$$

使用多项式多点求值可以在 $O(N \log^2 N)$ 的时间复杂度内解决。



杨表

Robinson-Schensted **双射**:

所有 1 到 n 的**排列**与所有**杨表标准 Tableau 二元组** (P, Q) 形成双射。

下面描述给出排列, 构造出 (P, Q) 的方法:

- 最初有一个空杨表 λ 。
- 按顺序向第一行插入排列中的元素。
- 往第 i 行插入元素 x 的规则如下:
 - 如果第 i 行没有元素比 x 大, 那么 x 将放在第 i 行末尾。
 - 否则找到最小的比 x 大的元素 y , 用 x 替换 y , 将 y 插入 $i + 1$ 行。

此时最后得到的杨表标准 Tableau 为 P 。

Q 与 P 形状相同, 但是 Q 中每个格子上的数字表示**这个格子的出现时间**。

具体可以参考[这个网站](#)。



杨表

设有一个排列 σ , 其对应杨表 λ 的两个标准 Tableau (P, Q) , 排列的第 i 个元素为 p_i 。

Robinson-Schensted **双射**具有如下性质:

- σ 的 LIS 长度为 λ 的列数, LDS 长度为 λ 的行数。
- 如 σ 与 (P, Q) 对应, 则 σ^{-1} 与 (Q, P) 对应。
- 如 $\sigma = \sigma^{-1}$, 则 $p_i = i$ 的个数等于 λ 中有奇数个格子的列的个数。
- $p_i > p_{i+1}$ **当且仅当** i 在 Q 中出现的行在 $i + 1$ 在 Q 中出现的行上方。



杨表

[ARC091 C] LISDL

Time Limit: 2 sec / Memory Limit: 256 MB

请构造一个长度为 N 的排列，满足 LIS 长度等于 A ，LDS 长度等于 B 。

无解输出 -1 。

数据范围

- $1 \leq A, B \leq N \leq 3 \times 10^5$



杨表

根据 Robinson-Schensted **双射**, 可以得到 $AB < N$ 与 $A + B - 1 > N$ 无解。

其余随便搞一个 P, Q 都是从上往下, 从左往右是 $1, 2, 3, \dots, N$ 的排列都行。

时间复杂度 $O(N)$ 。



杨表

[Yukicoder No.2048] L(I+D)S

Time Limit 2 sec / Memory Limit : 512 MB

求有多少个 LIS 长度 + LDS 长度 = N 的长度为 N 的排列。

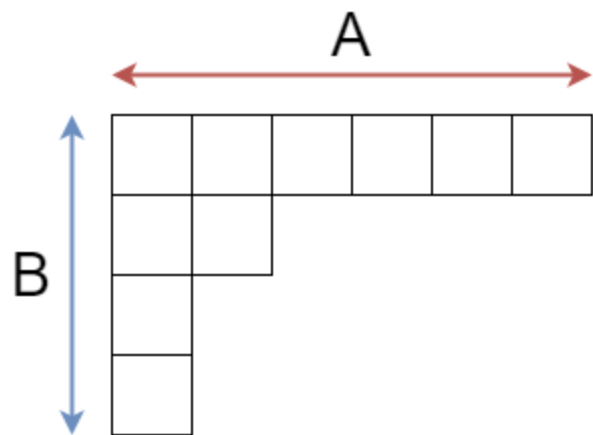
数据范围

- $1 \leq N \leq 10^5$



杨表

显然对于所有满足条件的排列，对应的杨表形状如下：



其中 $A, B \geq 2, A + B = N$ 。

直接使用钩长公式，可以得到标准 Tableau 数量为：

$$\frac{N!}{(A-2)!(B-2)!AB(N-1)}$$

直接枚举 A, B 的值加起来即可，时间复杂度 $O(N)$ 。



杨表

[ABC378 G] Everlasting LIDS

Time Limit: 4 sec / Memory Limit: 1024 MB

一个排列是好的当且仅当存在一个整数 n , 使得将 $n + 0.5$ 放到末尾后, LIS 和 LDS 长度都**不变**, 且 LIS 长度为 A , LDS 长度为 B 。

求长度为 $AB - 1$ 的好排列数量对 M 取模后的结果。

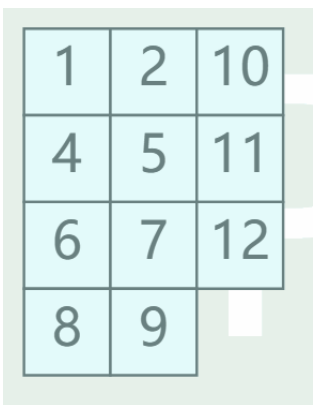
数据范围

- $2 \leq A, B$
- $AB < 120$
- $10^8 \leq M \leq 10^9$
- M 是质数。



杨表

考虑好的排列对应到的标准 Tableau P , 在插入 $n + 0.5$ 前后一定形如这样:
(若无法渲染, 点击 GIF 打开)



1	2	10
4	5	11
6	7	12
8	9	

即在插入前满足:

- 第一行末尾 $>$ 第二行倒数第二个
- 第二行末尾 $>$ 第三行倒数第二个
- ...

而对 Q 没有任何限制。



杨表

由于 $AB < 120$ ，最坏情况下，至多 A 行 B 列的杨表数量为 $\binom{22}{10} = 646646$ 。

直接对 P 进行 DP， Q 的方案数可以直接用钩长公式计算。

时间复杂度 $O(A \binom{A+B}{A})$ 。

该题直接用 `map` 存 `vector<int>` 可以通过。

在某些卡常的题目中，如果从形态上考虑，本质不同的转移不多，可以给杨表编号，然后预处理转移。

如果本质不同的形态和转移都比较多，那么只能按照字典序给杨表编号，然后尝试用组合数表示转移的状态间的编号之差。



伯努利数

我们考虑一个 m 次幂的求和式（这里认为 $0^0 = 1$ ）：

$$S_m(n) = \sum_{i=0}^{n-1} i^m = 0^m + 1^m + 2^m + \cdots + (n-1)^m$$

我们已知，对于 m 小的情况下，有如下的公式：

$$S_0(n) = n$$

$$S_1(n) = \frac{1}{2}n^2 - \frac{1}{2}n$$

$$S_2(n) = \frac{1}{3}n^3 - \frac{1}{2}n^2 + \frac{1}{6}n$$

$$S_3(n) = \frac{1}{4}n^4 - \frac{1}{2}n^3 + \frac{1}{4}n^2$$

$$S_4(n) = \frac{1}{5}n^5 - \frac{1}{2}n^4 + \frac{1}{3}n^3 - \frac{1}{30}n$$



伯努利数

实际上，对于任意 m ，其各项前面的系数是可以被表示出来的：

$$S_m(n) = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot n^{m+1-k}$$

其中 B_k 是**伯努利数**，其满足如下递推关系：

$$\sum_{j=0}^m \binom{m+1}{j} B_j = 0, (m > 0)$$

$$B_0 = 1$$



伯努利数

实际上，如果你不需要知道每一项具体的系数，有更加优秀的做法。

做法 1

考虑 $S_m(n)$ 是一个关于 n 的 $m + 1$ 次多项式。

可以直接使用拉格朗日插值，由于方便预处理 $n = 0, 1, \dots, m$ 的点值，所以预处理和单次插值都是 $O(m)$ 的。（使用线性筛，否则会带 $\log$ ）

做法 2

用有限微积分可以简单推得如下公式：

$$S_m(n) = \sum_{i=0}^m \left\{ \begin{matrix} m \\ i \end{matrix} \right\} \frac{(n+1)^{\underline{i+1}}}{i+1}$$

时间复杂度瓶颈在于预处理斯特林数，时间复杂度稍微劣于上一种。



排列计数 Trick

排列计数题是一类比较需要积攒 Trick 的题目，
由于排列**每个数恰好出现一次**的性质，
这类题目往往很难直接去计数，或设出 DP 状态。

然而正是因为这样，如果你普通的计数什么都不行，
那么这种题目的做法肯定就逃不出某几种特定的 Trick。

下面会通过例题提到几种经典的 Trick。



排列计数 Trick - ?

[EDPC T] Permutation

Time Limit: 2 sec / Memory Limit: 1024 MB

给你一个长度为 $N - 1$ 个字符串 S , 其中只包含 $<$ 和 $>$ 。

求满足下列条件的排列数量, 对 998244353 取模:

- 若 $S_i = <$, 则 $P_i < P_{i+1}$ 。
- 若 $S_i = >$, 则 $P_{i+1} < P_i$ 。

数据范围

- $1 \leq N \leq 3000$



排列计数 Trick - 插入计数 + 相对大小关系

设 $dp_{i,j}$ 表示确定了前 i 个数的**相对大小关系**,
且第 i 个数是前面这些数中第 j 小的。

转移考虑枚举第 $i + 1$ 个数可以成为前 $i + 1$ 个数中的
第 1 小, 第 2 小, $\dots$, 第 $i + 1$ 小, 前缀和优化转移即可。

时间复杂度 $O(N^2)$ 。



排列计数 Trick - 容斥

这题的解法很多，容斥也是其中一种方法。

我们考虑对于所有 $>$ 限制进行容斥，钦定若干个 $>$ 不满足，其余 $>$ 限制扔掉。

答案就是 钦定 0 个 $-$ 钦定 1 个 $+$ 钦定 2 个 $- \dots$

考虑被钦定的 $>$ 就相当于 $<$ ，而不被钦定的 $>$ 可以看作没有任何限制的 $\square$ 。

然后一段段 DP，每一段表示钦定后的一段连续的极长 $<$ ，
长度为 len ，中间夹了 $cnt_{>}$ 个 $>$ 的一段贡献为 $(-1)^{cnt_{>}} \times \frac{1}{len!}$ 。

注意由于要极长，所以如果当前位置右侧是 $<$ ，就不能进行转移。

由于一种分段对答案贡献为 $\binom{N}{len_1, len_2, \dots, len_k}$ ，所以最后要乘上 $N!$ 。

直接做时间复杂度 $O(N^2)$ ，用半在线卷积优化可以做到 $O(N \log^2 N)$ 。



排列计数 Trick - 插入计数 + 相对大小关系

如果放到树上，变成一条边的两个点有大小限制呢？

道——↗↘ 理~←→↑↓ 是一样的，设 $dp_{i,j}$ 表示 i 子树内的根是第 j 小的方案数。

合并某棵子树的过程考虑枚举根在合并前是第 j 小，合并时有 k 个放到前面来。

此时根前面合并有 $\binom{j-k-1}{k}$ 种方案，根后面合并有 $\binom{rootSize-j+sonSize-k}{sonSize-k}$ 种方案。

转移后当前根变为第 $j+k$ 小，然后子树那边根据限制的大小关系，把子树的根是相对第 $\leq k$ 小或者第 $> k$ 小的转移过来，前缀和优化即可。

时间复杂度 (N^2)。



排列计数 Trick - ?

[ARC148 E] $\geq K$

Time Limit: 2 sec / Memory Limit: 1024 MB

给你一个长度为 N 的正整数序列 $A = (A_1, A_2, \dots, A_N)$, 和一个正整数 K 。

求有多少个序列 A 的排列, 其满足任意相邻两项之和都 $\geq K$ 。

答案对 998244353 取模。

数据范围

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq A_i, K \leq 10^9$



排列计数 Trick - 插入计数 + 类折半报警器

注意到答案与原序列元素先后顺序无关，我们可以自己选择顺序去插入。

考虑把数字分为 $< \frac{K}{2}$ 和 $\geq \frac{K}{2}$ 两部分。

那么 $< \frac{K}{2}$ 不能连着放， $\geq \frac{K}{2}$ 连着放一定没事，两者之间的限制会比较微妙。

考虑从小到大插入每一个 $< \frac{K}{2}$ 的数，按照下面的规则插入：

- 假设当前插入第 i 小的值 val ，在这之前先把所有 $\geq K - val$ 的值插入。
- 将 val 插入。
- 上述两次插入操作都不能和已插入的 $< \frac{K}{2}$ 的数相邻，
而由于 $< \frac{K}{2}$ 的数之间不会相邻，所以可以插的位置数可以直接计算出来。

时间复杂度 $O(N \log N)$ ，瓶颈在于排序。



排列计数 Trick - 插入计数

这种 Trick 用到的是最多的，所以你可以优先考虑这个能不能做。

光是这种 Trick 就有**按照位置从小到大插，按照位置从大到小插，按照值从小到大插，按照值从大到小插，按照某种奇怪的顺序插**等具体的方式，对于该 Trick 怎么用还是需要好好想想的。

时间复杂度主要以 $O(N^2)$ 为主，情况简单也有 $O(N)$ 的。

要记录的状态比较多会往 $O(N^3)$, $O(N^4)$ 上面走，但是比较少见。



排列计数 Trick - ?

Time Limit: 2 sec / Memory Limit: 1024 MB

给你一个长度为 N 的 `01` 串，每次删掉一个元素然后左右两边拼起来，求有多少种本质不同的删除方案，答案对 998244353 取模。

两种删除方案本质不同，当且仅当存在一个 i ，使得第 i 次删除后，两种删除方案剩下的串不同。

数据范围

- $1 \leq N \leq 500$



排列计数 Trick - 区间 DP + 钦定方法使不数重 + 时光倒流

考虑对于每个连续段，里面删除哪个元素都是一样的，
所以我们钦定删除的元素**左侧的元素不能和自己相同**。

然后时光倒流，变成每次插入某个元素时，其左侧的那个元素不能和自己相同。

考虑往某个位置插入元素后，该位置左右两边之间就互不影响了，是区间 DP 的形式。

那么考虑一个子区间的某个位置放入元素，该元素不能和区间左端点左侧的元素相同。

可以写出 DP 转移式子：

$$dp_{l,r} = \sum_{k \in [l,r], s_k \neq s_{l-1}} dp_{l,k-1} \cdot dp_{k+1,r} \cdot \binom{r-l}{k-l}$$

时间复杂度 $O(N^3)$ 。



排列计数 Trick - 区间 DP

对于排列的限制恰好构成区间 DP 形式的，可以用区间 DP 解决。

往往你需要对整个题目形式进行一定的分析才行。

时间复杂度往往是 $O(N^3)$ ，形式简单的会是 $O(N^2)$ 。

如果需要记录更多信息，那么 $O(N^4)$, $O(N^5)$ 也会有，这种题就比较繁杂了。

前面那道例题其实你要敏锐注意到你要数的是删除位置的一个排列，也就是意味着很多非常 Naive 的 DP 多半是假的，会数漏数重。



排列计数 Trick - ?

[ABC134 F] Permutation Oddness

求长度为 N , $\sum_{i=1}^N |i - P_i| = K$ 的排列数量。

数据范围

- $1 \leq N \leq 50$
- $0 \leq K \leq N^2$



排列计数 Trick - 二分图匹配

考虑 N 个上部点和 N 个下部点匹配, 上方第 i 个与下方第 j 个点匹配表示 $P_i = j$ 。

此时 $\sum_{i=1}^N |i - P_i|$ 就容易刻画了, 其等于每对匹配边横跨的距离。

然后考虑从左往右, 考虑当前的上下两个点的匹配情况:

- 直接上下连接。
- 两个都留给后面。
- 其中一个和左边匹配, 一个留给后面。
- 两个都和前面匹配。

设 $dp_{i,j,k}$ 表示, 考虑到第 i 对点, 前面留了 j 对点未匹配, 匹配总横跨距离为 k 。

直接 DP, 时间复杂度 $O(N^4)$ 。



排列计数 Trick - 二分图匹配

顺带一提，逆序对数也可以这么刻画，其表示匹配边交叉的次数。

比如是某些位置的值已知的排列，你就不能用插入 DP 来刻画排列逆序对了。

这种 DP 能刻画的东西会比较特别，别的可以具体根据题目内容分析。

时间复杂度一般 $O(N^4)$ 起步，因为维护的状态一般会比较大。

当然不排除某些只维护奇偶性的，时间复杂度会小。



排列计数 Trick - ?

[HDU 7525] cats 的凸包计算

Time Limit: 16 sec (HDU 的机子速度, 谅解一下) / Memory Limit: 512 MB

定义一个排列 $P = (P_1, P_2, \dots, P_N)$ 的面积为:

- 在平面直角坐标系上, $(1, P_1), (2, P_2), \dots, (N, P_N)$ 这 N 个点的凸包的面积。

有一个长为 N 的序列 $A = (A_1, A_2, \dots, A_N)$,

每个数均为一个 1 到 N 间的正整数或 -1 。

求将所有 -1 替换成 1 到 N 之间的正整数后, 形成的所有排列的面积之和。

答案对 $10^9 + 7$ 取模。

数据范围

- $1 \leq N \leq 50$
- $A_i = -1 \vee 1 \leq A_i \leq N$



排列计数 Trick - 拆贡献

多边形面积为 $\frac{\sum_{i=1}^N (y_i + y_{i+1})(x_i - x_{i+1})}{2}$ ，也就是若干个拉到 x 轴的梯形相加减。

也就是说，面积可以拆到多边形的每条边上计算。

对于点集的凸包，可以看作**上凸壳的边贡献** — **下凸壳的边贡献**。

这里只考虑上凸壳贡献，下凸壳同理。

直接 $O(N^4)$ 枚举排列两个位置以及填什么值，将其作为一条上凸壳的边。

那么其它的点都要在这条边所在直线下方，方案数可以 $O(N)$ 计算。

细节：凸壳上的边会三点共线，所以钦定两点之间的不能与直线重合，两边可以重合。

总时间复杂度 $O(N^5)$ 。



排列计数 Trick - ?

[ABC330 G] Inversion Squared

有一个长为 N 的序列 $A = (A_1, A_2, \dots, A_N)$,
每个数均为一个 1 到 N 间的正整数或 -1 。

求将所有 -1 替换成 1 到 N 之间的正整数后, 形成的**排列的逆序对数的平方**之和。

答案对 998244353 取模。

数据范围

- $1 \leq N \leq 3000$
- $A_i = -1 \vee 1 \leq A_i \leq N$



排列计数 Trick - 组合意义/拆贡献

直接组合意义，枚举两对数钦定是逆序对，然后数方案。

这题恶心在你需要做到 $O(N^2)$ ，所以只要小小分讨几十种情况，分别写出一个实现优秀的计算方法即可。



排列计数 Trick - 拆贡献

有些题目你直接 DP 排列完全不可做。

尤其是给你一个**有些位置已知，有些位置是 -1 的序列**的类型，

能做这个的 DP 已知的可能只有前面的二分图匹配。



排列计数 Trick - 杨表

排列要用到杨表那就是 RSK 双射了。

前面讲过例题了，这里不赘述。



排列不一定是计数的 Trick - ?

[CF1458C] Latin Square

给你一个 $N \times N$ 的拉丁方。（每行每列都是一个 1 到 N 的排列）

你需要维护 M 次操作，包含的 6 种操作规则如下：

- **U / D**：每列向上/下循环位移一位。
- **L / R**：每行向左/右循环位移一位。
- **C**：每列的排列变成其的逆排列。
- **I**：每行的排列变成其的逆排列。

数据范围

- $1 \leq N \leq 2000$
- $1 \leq M \leq 10^6$



排列不一定是计数的 Trick - 逆排列的本质

对于一个排列，其本质上是 N 个二元组信息 $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$ ，信息 (x_i, y_i) 表示 $P_{x_i} = y_i$ 。

而对于 P^{i-1} 来说，信息 (x_i, y_i) 表示 $P^{-1}_{y_i} = x_i$ 。

对于拉丁方也是一样的，如果用信息 (x_i, y_i, z_i) 表示 $A_{x_i, y_i} = z_i$ ，那么 6 种操作分别可以这样表示：

- **U / D**：令所有信息 $(a, b, c) \rightarrow (a \pm 1, b, c)$ 。
- **L / R**：令所有信息 $(a, b, c) \rightarrow (a, b \pm 1, c)$ 。
- **C**：令所有信息 $(a, b, c) \rightarrow (c, b, a)$ 。
- **I**：令所有信息 $(a, b, c) \rightarrow (a, c, b)$ 。

所以只要设三个元构成信息 (x, y, z) 去执行所有操作，然后把 N^2 个信息分别代入 x, y, z 后就得到了最后的拉丁方。



排列不一定是计数的 Trick - 逆排列的本质

该 Trick 虽然不一定用于计数，但是在计数时还是可以用到的。

有的时候对于原排列不好 DP，可以考虑其逆排列，即位置和价值域交换。

对于某些题意在这样的转换后，也会变得更好分析。

但是很多题目给你的形式本身反而是更好分析的，转化需谨慎，也不要长时间陷入在转化后的题意中（对于任何题意的转化都是如此）。



GL & HF